```
# declare the GP
lengthscale = 1
noise = 1e-4

likelihood = gpytorch.likelihoods.GaussianLikelihood()
model = BaseGPModel(train_x, train_y, likelihood)

# fix the hyperparameters
model.covar_module.lengthscale = lengthscale
model.likelihood.noise = noise

model.eval()
likelihood.eval()
```

4. To assemble our test data set, we first create a mesh grid for the first and second columns that spans over the unit square.

```
grid_x = torch.linspace(0, 1, 101)

grid_x1, grid_x2 = torch.meshgrid(grid_x, grid_x, indexing="ij")
```

These first two columns are stored in `grid_x1` and `grid_x2`. We then append two additional, all-zero columns to `grid_x1` and `grid_x2`, completing test set with four columns.

```
xs = torch.vstack(
    [
        grid_x1.flatten(), ❶
        grid_x2.flatten(), ❷
        torch.zeros(101 ** 2), ❸
        torch.zeros(101 ** 2), ❹
    ]
).transpose(-1, -2)
```

❶ First column.

❷ Second column.

❸ Third column containing all zeros.

❹ Forth column containing all zeros.

5. To make predictions on this test set, we simply pass `xs` through our GP model under the `torch.no_grad()` context.

```
with torch.no_grad():
    predictive_distribution = likelihood(model(xs))
    predictive_mean = predictive_distribution.mean
    predictive_stddev = predictive_distribution.stddev
```

6. To visualize these predictions, we first create a figure with two panels (that is, two Matplotlib subplots).

```
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
```

We then use `plt.imshow()` to visualize the mean and standard deviation vectors as heat maps, making sure to reshape the two vectors into square matrices

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>